

KFOLD: Scalable Object-Selection Conditions for Kbuild Makefiles

ANONYMOUS AUTHOR(S), Anonymous Institution, USA

Kbuild Makefiles decide which objects enter the build of Linux and other configurable systems. A maintainer who asks why a file was not compiled, or which options would compile it, needs the option condition under which each object is built, but running Make reveals only the objects of one configuration. Computing these conditions statically requires following assignments whose destination depends on an option, overwrites that apply in only some configurations, composite objects, included fragments, directories reached along several paths, and objects that Kbuild builds only because a rule needs them. Executing a Makefile symbolically with one state per branch combination does not scale: on a generated Makefile with 16 independent options, a forking executor tracks more than half a million states. We present KFOLD, which executes Kbuild Makefiles with a Boolean guard on every word of every variable. Appends add guarded words, a conditional overwrite re-guards the words it replaces, and the two arms of a conditional merge into one state at `endif`, so the state stays linear in the number of words. A directory worklist propagates to each Makefile only the part of an incoming guard not already covered, and a closure over rules adds the objects that selected files depend on.

We evaluate KFOLD on Linux 7.2.8 and four other Kbuild-based systems by evaluating its conditions under concrete configurations and comparing the predictions with the objects of fresh builds. On four Linux configurations, from `tinyconfig` to `allmodconfig`, every object that KFOLD predicts is compiled and every compiled object that it extracts is predicted: 512 to 29,556 objects, with no false positive and no miss. The same holds for the other four systems. KFOLD analyzes the 2,215 Makefile instances of the Linux x86 tree in 143.1 s and 203.9 MB, and its time stays flat on generated Makefiles where the forking executor times out. Its conditions explain the build status of all 400 sampled Linux objects and, combined with a Kconfig model, yield verified configurations for 384 of the 402 commits of a Linux stable update that touch C or assembly files. These results show that per-word guards, together with a faithful model of Kbuild's build semantics, yield object-selection conditions that are both scalable and accurate at kernel scale.

CCS Concepts: • **Software and its engineering** → **Software verification**.

Additional Key Words and Phrases: Kbuild, build systems, variability, symbolic execution, object selection

1 Introduction

The build system determines which source files reach the compiler. In the Linux kernel and related projects, Kbuild expresses this choice through Make variables such as `obj-$(CONFIG_USB)` and through conditionally selected subdirectories. A concrete build reveals the objects selected by one `.config`; it does not say which option settings would select a different object. A build condition for every object would let a maintainer explain why a source file was not compiled, compare build rules across releases, or search for a configuration that enables a given target. Analyses of configuration-dependent compilation errors and unmet dependencies already consume such conditions [5, 14].

Recovering these conditions requires more than collecting the `CONFIG_*` names that appear near an object. Consider two consecutive lines of a Kbuild Makefile:

```
obj-$(CONFIG_NET) += net_core.o    obj-$(CONFIG_FAST) := fast_net.o
```

The first line appends `net_core.o` to `obj-y` when `CONFIG_NET=y`. The second computes its destination from `CONFIG_FAST`: when that option is `y`, it *replaces* `obj-y`; otherwise it assigns an unselected variable such as `obj-` and leaves `obj-y` alone. The correct condition for `net_core.o` is therefore

Author's Contact Information: Anonymous Author(s), Anonymous Institution, USA.

2026. ACM XXXX-XXXX/2026/9-ART
<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

`CONFIG_NET` $\wedge$ $\neg$ `CONFIG_FAST`. Further conditions come from `ifeq` blocks, from composite objects whose members are listed in other variables, from fragments included with `include`, from the chain of parent directories through which Kbuild reaches a Makefile, and from rules: the Linux boot image, real-mode trampoline, and vDSO consist of objects that no target list names and that Kbuild builds only because a rule needs them.

Symbolic execution is a natural way to compute such conditions, because it evaluates Make’s assignments and expansions as written while tracking the configurations under which each result holds. Its classic form keeps one state per path. An earlier symbolic executor for Kbuild Makefiles [13] forks a state at each conditional and merges only identical paths. Kbuild Makefiles, however, consist largely of independent conditional assignments, and each one that changes a value can double the number of distinct states. On a generated Makefile with 16 independent `ifeq` blocks that each append to a helper list, that executor tracks 524,289 states and needs 187.1 s; with 20 blocks it does not finish within five minutes (Section 5.3). The Linux x86 tree has 2,215 Makefile instances with thousands of such assignments.

KFOLD starts from the observation that Kbuild object lists are used as *sets*: an object is either in `obj-y` or it is not. KFOLD therefore attaches a guard to each word rather than keeping one state per configuration. In the example, `obj-y` holds `net_core.o` under `CONFIG_NET` $\wedge$ $\neg$ `CONFIG_FAST` and `fast_net.o` under `CONFIG_FAST`. A conditional overwrite conjoins the negation of its own guard onto the words it may replace. For an `ifeq` block, KFOLD executes each arm on a copy of the state and merges the copies at `endif`, so one state continues after every conditional and k guarded appends yield k guarded words. Across directories, a worklist records for each Makefile the disjunction of guards already propagated to it and processes only the uncovered part of a new guard. On top of this representation, KFOLD models the parts of Kbuild’s build semantics that decide which object files exist: composite members, the rule that built-in objects need a built-in route to their directory, host and user programs, and a closure over rules and sub-makes.

A scalable representation is useful only if its conditions are right, and the strongest evidence for that is agreement with real builds. We therefore evaluate KFOLD’s conditions under concrete configurations against the `.o` files that a fresh build of each configuration produces. The subjects are the current releases of Linux, under four configurations, and of BusyBox, Barebox, Das U-Boot, and coreboot. On all four Linux configurations, every predicted object is compiled and every compiled object that KFOLD extracts is predicted, from 512 objects for `tinyconfig` to 29,556 for `allmodconfig`; the compiled objects that it does not extract are host tools. KFOLD analyzes the Linux x86 tree in 143.1 s and 203.9 MB. On the same tree, KMAX [3], a static analysis that also carries configuration conditions on values, needs 6,275 s and 27.9 GB and misses up to 9,038 compiled objects that it extracts. Built on these conditions, KFOLD’s `why` command explains the build status of every sampled object, and its `config-for` command produces verified configurations for the files that real stable-kernel patches touch.

This paper makes the following contributions:

- (1) A per-word guarded representation of Make variables, with guarded append, conditional overwrite, and branch merge operations for Kbuild-style assignments with computed names, and an expansion that preserves Make’s semantics for word lists, partially defined variables, substitution references, and deferred appends (Sections 3.3 to 3.5).
- (2) A conditional directory worklist that propagates only uncovered guards, static splicing of included fragments, and a model of Kbuild’s build semantics, including built-in routes and a closure over rules and sub-makes (Sections 3.2, 3.6 and 3.8).
- (3) KFOLD, an implementation that supports the target families, composite objects, and top-level entry points of five Kbuild dialects, and commands that explain an object’s build status, find

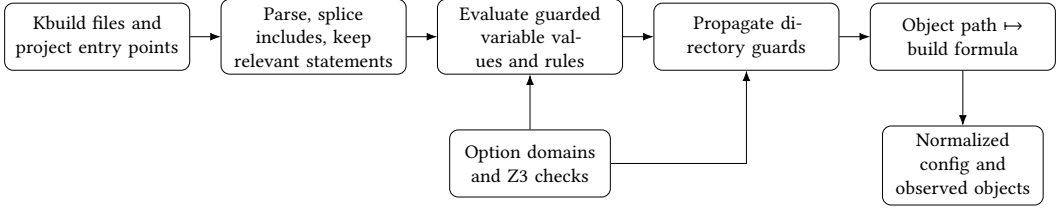


Fig. 1. KFOLD computes one build formula per object path without choosing a configuration. A normalized configuration and the observed object inventory enter only afterward, to evaluate and validate the formulas.

configurations for a patch, and list code that no given configuration builds (Sections 3.7 and 4).

- (4) An evaluation against fresh builds of five systems, with no false positive and no in-set miss on any configuration, a scalability comparison with a forking symbolic executor and with KMAX, an evaluation of the commands on sampled objects and real patches, and an ablation of each design element (Section 5).

2 Overview

Figure 1 shows how KFOLD turns Kbuild files and project entry points into a formula for each object path. KFOLD first parses each Makefile, splices in the fragments it includes, and removes statements that cannot affect target variables or rules. It then executes the remaining statements over guarded variable values, propagates directory guards from parent to child Makefiles, and finally collects guarded object paths: targets, composite members, program objects, and the prerequisites that rules need. A separate evaluator substitutes a concrete configuration into the formulas and compares the predictions with observed objects. The next subsections follow one Makefile through these stages; Section 3 generalizes each operation.

2.1 Kbuild selection in brief

Kbuild uses GNU Make variables to describe objects. A line such as `obj-$(CONFIG_NET) += net.o` computes the destination variable name from an option. If the option expands to `y`, the word is appended to `obj-y` and linked into the kernel; `m` selects `obj-m` for a loadable module; an unset option produces `obj-`, which Kbuild ignores. A word ending in `/` asks Kbuild to descend into that subdirectory’s Makefile. A target such as `driver.o` may be a composite assembled from the objects in `driver-y` or `driver-objs`, and a Makefile may pull further assignments from another file with `include`. The condition for an object therefore combines a local selection condition with a reachability condition inherited from its parent directories.

GNU Make also distinguishes immediate assignment with `:=` from deferred assignment with `=`. The former expands its right-hand side at assignment time; the latter stores an expression that is expanded when the variable is referenced. `+=` follows the variable’s existing flavor, and `?=` assigns only when the variable is undefined. A reference to a variable that is not set in the current configuration expands to the empty string. These rules decide which assignment wins and what a reference produces, and so affect object lists even when a Makefile contains no explicit conditional.

KFOLD takes a source tree, per-project settings that name the entry directories and target-variable families, and a Boolean or tristate option domain. It outputs a set of object paths, each with a formula over those options.

Listing 1 is a simplified Kbuild Makefile that extends the two lines from Section 1. It combines an assignment to a computed variable name, an overwrite, a conditional branch, and a composite

Listing 1. Simplified Kbuild Makefile. Each object’s final condition also includes the guard under which this directory is entered.

```

1 obj-$(CONFIG_NET) += net_core.o
2 obj-$(CONFIG_FAST) := fast_net.o
3 ifeq ($(CONFIG_WIDE),y)
4     WIDTH := 64
5 else
6     WIDTH := 32
7 endif
8 obj-y += driver_$(WIDTH).o
9 fast_net-y += accelerator.o

```

Table 1. Guarded words after each statement of Listing 1. The last column assumes that the directory is reached under guard D . The map records membership, not word order or multiplicity.

Line	Step	State change	Final contribution
1	Append	<code>net_core.o</code> : N	$D \wedge N \wedge \neg F$
2	Conditional overwrite	<code>fast_net.o</code> : F ; <code>net_core.o</code> : $N \wedge \neg F$	$D \wedge F$
3–7	Conditional	WIDTH: 64 under W , 32 under $\neg W$	—
8	Append	<code>driver_64.o</code> : W	$D \wedge W$
8	Append	<code>driver_32.o</code> : $\neg W$	$D \wedge \neg W$
9	Composite	<code>accelerator.o</code> belongs to <code>fast_net.o</code>	$D \wedge F$

target. For this example, assume that each option is either `y` or unset. We want a condition for each object, not a list for one chosen configuration.

2.2 A trace from assignments to object conditions

Table 1 records the state after each statement of Listing 1. We write N for `CONFIG_NET=y`, F for `CONFIG_FAST=y`, and W for `CONFIG_WIDE=y`. Line 1 adds one word, `net_core.o`, with guard N . Line 2 assigns `obj-y` only when F holds, so `KFOLD` adds `fast_net.o` under F and re-guards the existing word: `net_core.o` survives under $N \wedge \neg F$. Treating the overwrite as unconditional would lose `net_core.o`; ignoring it would keep `net_core.o` under $N \wedge F$, where `Make` has replaced the value.

Lines 3–7 assign `WIDTH` in both arms of a conditional. `KFOLD` executes the then-arm on one copy of the state under W and the else-arm on another under $\neg W$. At `endif`, it merges the copies: `WIDTH` holds 64 under W and 32 under $\neg W$, and execution continues with one state. Line 8 expands `driver_$(WIDTH).o` into two words with complementary guards. Line 9 defines a composite: its parent `fast_net.o` is selected under F , so its unconditional member `accelerator.o` inherits F . Finally, `KFOLD` conjoins every local condition with the guard D under which the directory is reached, which gives the last column of Table 1.

A forking executor would instead carry one environment for each combination of N , F , and W that produces a different `obj-y`, and every further independent conditional that changes the list can double that number. `KFOLD`’s state after line 9 holds five guarded object words, and a further independent append adds one more.

2.3 The same object through two directories

Directory guards need the same care as variable guards. Suppose a root Makefile selects `a/` under option A and `b/` under option B , and both directories select `shared/`. If `shared/Makefile` selects

$x.o$, the condition for $x.o$ is $A \vee B$. Marking `shared/` as visited after the A path would lose the B contribution. `KFOLD` instead records, for each Makefile, the disjunction of the guards already propagated to it. When B arrives, it processes only $B \wedge \neg A$; if A already covered B , it does no new work. For the same reason, a back edge from `shared/` to `a/` stops once it contributes no new configuration.

3 Technique

3.1 Setting and algorithm

Let M be a collection of Kbuild Makefiles and c an assignment to the option symbols they use. For an object path o , write $S_M(o, c)$ when Kbuild builds o under c . `KFOLD` computes a formula $\Phi_M(o)$ such that evaluating $\Phi_M(o)$ under c predicts $S_M(o, c)$. A variable's state $\sigma[n]$ is a map from words to formulas: an entry $w \mapsto \phi$ says that w belongs to variable n in configurations satisfying ϕ . We write $\sigma[n](w) = \perp$ for an absent word and $\sqcup$ for the pointwise disjunction of two such maps.

Algorithm 1 gives the end-to-end procedure. Its input is the source tree, the entry Makefiles E with their guards, and project settings Γ that name target-variable families and option domains; its output is the map Φ from object paths to formulas. Lines 3–8 take a Makefile m with incoming guard γ and keep only the part δ not already covered by $R[m]$. Lines 9–11 parse m , splice its includes, reduce it, and execute it once with `EXEC` (Algorithm 2); line 12 restricts the cached state to δ , and line 13 records the restricted instance. Lines 14–16 queue the child directories selected in that instance. After the queue empties, lines 18–23 collect targets, composite members, and program objects from all instances and disjoin repeated contributions, and line 24 adds the objects that rules need (Section 3.8). A missing entry file contributes no edge, and a file that fails to parse contributes an empty state and is reported.

Algorithm 2 executes one statement s on state σ under ambient guard g ; a statement list is executed in order. Lines 1–10 handle an assignment: line 2 expands the right-hand side into guarded words, line 3 expands the left-hand side into guarded variable names, and lines 5–9 either overwrite or append. Lines 11–16 handle a conditional: line 12 evaluates the test and copies the state, line 13 executes both arms on the copies, and lines 14–16 merge the copies back into σ . Rules and sub-make recipes are recorded as guarded lists as well (Section 3.8); include directives have been replaced by the statements they name.

The two algorithms connect through the overview example. For Listing 1, $C[m]$ holds the two guarded `WIDTH` words; restricting that state by the parent guard D at line 12 of Algorithm 1 produces $D \wedge W$ and $D \wedge \neg W$ for the two driver objects. In the shared-directory example, `shared/Makefile` is processed first under A and then under $B \wedge \neg A$, and line 21 yields $A \vee B$ for $x.o$.

The observation unit is an object path, not a source file. A path can denote a compiler output, a module's composite object, a member of a composite, an object of a host program, or a link artifact. Section 5 maps a path to a physical object when the normalized spellings match.

3.2 Parsing, include splicing, and dependence reduction (Algorithm 1, line 10)

`KFOLD` reads a Makefile, normalizes line endings, and parses it with `pymake3`. Kbuild evaluates an included fragment in the context of the including Makefile, with the source tree as working directory. `KFOLD` mirrors this by *splicing*: it replaces each include whose path it can resolve with the statements of the named file, recursively and without cycles, before any further processing. Paths are resolved statically from `src`, `obj`, `src tree`, the configured fixed definitions, earlier assignments in the same file whose values are constant, and `addprefix` and `addsufffix` calls over resolved arguments. The Linux AMD GPU driver, for example, defines `FULL_AMD_PATH := $(src tree)/$(src)/..`, includes `$(FULL_AMD_PATH)/amdkfd/Makefile`, and builds the list of

Algorithm 1 Extract guarded Kbuild object conditions

Require: source tree M , guarded entry files E , project settings Γ

Ensure: object-path map Φ

```
1:  $R[m] \leftarrow \perp$  for each file  $m$ ;  $Q \leftarrow E$ 
2:  $C \leftarrow$  empty map;  $I \leftarrow \emptyset$ 
3: while  $Q$  is nonempty do
4:   remove  $(m, \gamma)$  from  $Q$ 
5:    $\delta \leftarrow \gamma \wedge \neg R[m]$ 
6:   if  $\delta$  is unsatisfiable then
7:     continue
8:   end if
9:   if  $C[m]$  is undefined then
10:     $C[m] \leftarrow \text{EXEC}(\text{reduce}(\text{splice}(\text{parse}(m))), \sigma_\Gamma, \Gamma)$ 
11:   end if
12:    $S \leftarrow \text{restrict}(C[m], \delta)$ 
13:    $R[m] \leftarrow R[m] \vee \gamma$ ; add  $S$  to  $I$ 
14:   for each selected directory  $(d, \eta)$  in  $S$  do
15:     add  $(\text{entry}(d), \eta)$  to  $Q$  if its file exists
16:   end for
17: end while
18:  $\Phi \leftarrow \emptyset$ 
19: for each restricted instance  $S \in I$  do
20:   for each target, member, or program object  $(o, \phi) \in \text{objects}(S, \Gamma)$  do
21:      $\Phi[o] \leftarrow \Phi[o] \vee \phi$  (initially  $\perp$ )
22:   end for
23: end for
24:  $\Phi \leftarrow \text{close}(\Phi, I, \Gamma)$  ▷ rule prerequisites and sub-makes
25: return  $\Phi$ 
```

display-library Makefiles to include with `addprefix` and `addsufffix`; after splicing these 51 files, the composite `amdgpu.o` has 786 members. An include whose path remains unresolved stays in place and contributes nothing, and `KFOLD` counts it.

A preliminary dependency pass then identifies the variables that can influence target families, composites, programs, and rules, and removes the statements that cannot. `EXEC` runs on the reduced list against a fresh state σ_Γ that holds `src`, `obj`, and the configured definitions. The reduction keeps production Makefiles tractable, because most of their statements set compiler flags that do not affect which objects exist. A file that cannot be parsed contributes an empty state and is reported; in the evaluation this occurs for one `coreboot` file.

3.3 Guarded assignment (Algorithm 2, lines 1–10)

Consider an assignment whose left-hand side expands to name n under guard ψ (lines 3–4). An append adds each expanded right-hand word under the conjunction of ψ and the word’s own expansion guard (line 8). An overwrite replaces n where ψ holds: new words receive $\psi \wedge \phi_w$, and an old word guarded by ϕ survives under $\phi \wedge \neg\psi$ (line 6). If an old and a new contribution share a word, $\sqcup$ disjoins their guards. Line 2 of Listing 1 exercises this case: $\psi = F$, so `net_core.o` goes from N to $N \wedge \neg F$. The same operation handles a guard that comes from the computed name rather than from an enclosing `ifeq`.

Algorithm 2 EXEC: execute one statement over guarded words

Require: statement s , state σ , ambient guard g

Ensure: updated state σ

```
1: if  $s$  is an assignment  $x \text{ op } e$  then
2:    $V \leftarrow \{w \mapsto \phi_w\}$  from  $\text{expand}(e, \sigma)$ , or  $\{e \mapsto \top\}$  if  $\text{op}$  defers  $e$  and  $e$  has a reference
3:   for each guarded name  $(n, \psi_n) \in \text{expand}(x, \sigma)$  do
4:      $\psi \leftarrow g \wedge \psi_n$ 
5:     if  $\text{op} \in \{=, :=\}$  or  $\sigma[n]$  is undefined then
6:        $\sigma[n] \leftarrow \{w \mapsto \psi \wedge \phi_w\} \sqcup \{w \mapsto \phi \wedge \neg\psi : (w \mapsto \phi) \in \sigma[n]\}$ 
7:     else
8:        $\sigma[n] \leftarrow \sigma[n] \sqcup \{w \mapsto \psi \wedge \phi_w\}$ 
9:     end if
10:  end for
11: else if  $s$  is a conditional with test  $t$ , then-arm  $T$ , else-arm  $F$  then
12:    $\kappa \leftarrow \text{cond}(t, \sigma)$ ;  $\sigma_t \leftarrow \text{copy of } \sigma$ ;  $\sigma_e \leftarrow \text{copy of } \sigma$ 
13:   EXEC( $T, \sigma_t, g \wedge \kappa$ ); EXEC( $F, \sigma_e, g \wedge \neg\kappa$ )
14:   for each name  $n$  assigned in  $T$  or  $F$  do
15:      $\sigma[n](w) \leftarrow g \wedge \phi$  if  $\sigma_t[n](w) = \sigma_e[n](w) = \phi$ , else  $(g \wedge \kappa \wedge \sigma_t[n](w)) \vee (g \wedge \neg\kappa \wedge \sigma_e[n](w))$ 
16:   end for
17: end if
18: return  $\sigma$ 
```

KFOLD follows GNU Make's flavors. For $:=$ it expands the right-hand side at assignment time. For $=$ and $?\equiv$, and for $+=$ on a variable that is undefined or recursive, it stores text that contains a reference for later expansion and splits text without references into words directly (line 2); Linux's kernel/Makefile, for instance, starts `obj-y = fork.o exec_domain.o . . .`. When a Makefile has been executed, KFOLD expands the deferred words of its target, composite, and program lists, because Kbuild reads those lists only after the whole file. The difference is visible in `drivers/xen/Makefile`, which appends `$(xen-pad-y)` to `dom0-y` one line *before* it defines `xen-pad-y`; only the deferred reading finds `xen-acpi-pad.o`. Line 5 treats only $=$ and $:=$ as overwrites, so $?\equiv$ and $:=$ on an already defined variable take the append path; GNU Make would instead leave the value unchanged for $?\equiv$ and replace it for $:=$. Table 2 summarizes these boundaries.

3.4 Computed names and expansion (Algorithm 2, lines 2–3)

In `obj-$(CONFIG_X) += foo.o`, the left-hand side expands to `obj-y`, `obj-m`, or an unselected name under distinct guards. EXEC expands the name before updating the state (line 3) and combines the name guard, ambient guard, and value guard (lines 4, 6, and 8). A single assignment can therefore contribute to several target families. The same expansion produces `driver_64.o` and `driver_32.o` from `driver_$(WIDTH).o` in Listing 1.

An assignment's right-hand side is a *word list*: parts separated by whitespace are independent elements, and only parts written without whitespace between them, such as `driver_$(WIDTH).o`, are concatenated by taking the product of their alternatives. Linux's `arm_scmi` Makefile, for example, defines `scmi-module-objs := $(scmi-driver-y) $(scmi-protocols-y) $(scmi-transport-y)`; treating the value as one concatenation would multiply the three lists' alternatives, while the word-list reading keeps each object's own guard. Names, `ifeq` operands, and function arguments are expanded as strings.

Table 2. Make constructs and `kFOLD`’s interpretation.

Construct	Implemented behavior	Boundary
<code>+=, =, :=</code>	Guarded words; GNU Make flavors, deferred lists expanded after the file	Order and duplicates are lost.
<code>?=, ::=</code>	Assign if undefined; otherwise append	Differs from GNU Make when the variable is defined.
<code>ifeq, ifneq, ifdef, ifndef</code>	Copy arms, execute, re-guard, merge	—
References, $\$(V:a=b)$	Guarded words plus the empty alternative; substitution per word	—
Text functions and <code>call</code>	Guarded alternatives	Concatenations above 64 alternatives are collapsed.
<code>include</code>	Spliced when the path resolves statically	Unresolved paths contribute nothing.
Rules and sub-makes	Prerequisites followed from selected files	Recipes other than sub-makes are not executed.
$\$(shell \dots)$	Concrete subprocess (none in sandbox mode)	Host environment may affect results.
$\$(eval \dots)$	Each guarded expansion parsed and executed in place	Unparsable text contributes nothing.

A reference to a variable expands to its guarded words and, where none of them is present, to the empty string. The empty alternative matters inside a longer value. The Makefile of Linux’s `dvb-core` driver defines `dvb-net-$(CONFIG_DVB_NET) := dvb_net.o` and then `dvb-core-objs := dvbdev.o . . . $(dvb-net-y) . . .`, and without it every member of the composite would depend on that option. Substitution references such as $\$(IPA_VERSIONS:\%=data/ipa_data-v\%.o)$ and `ixgbe-$(CONFIG_FCOE:m=y)` apply GNU Make’s pattern substitution to each guarded word. Concatenation drops combinations whose guard simplifies to false, and when more than 64 alternatives remain, it collapses them into one string under an unconditional guard. The expander also handles substitution, filtering, prefix and suffix addition, string tests, positional words, `foreach`, and `call`. A $\$(shell \dots)$ expression runs as a concrete subprocess with a five-second timeout, so the analysis depends on the host shell and tools; the evaluated Linux tree contains 15 such calls.

3.5 Conditional execution and branch merge (Algorithm 2, lines 11–16)

For `ifeq` and `ifneq`, the condition evaluator expands both sides into guarded alternatives; equality holds under the disjunction of the guards of equal pairs (line 12). For `ifdef` and `ifndef`, it checks under which guards the variable’s expansion is undefined. A chain of `else ifeq` arms is a nested conditional in the `else-arm`, so only the first matching arm contributes.

`EXEC` copies the entering state for each arm and executes the copies under $g \wedge \kappa$ and $g \wedge \neg \kappa$ (line 13). The merge at lines 14–16 rebuilds only the variables assigned in either arm. For such a variable, a word with guard ϕ_t in the then-copy and ϕ_e in the else-copy receives $(g \wedge \kappa \wedge \phi_t) \vee (g \wedge \neg \kappa \wedge \phi_e)$. Re-guarding matters for a word that an arm did not touch: suppose `WIDTH` had the value 16 before line 3 of Listing 1. Each copy still carries 16 after its own overwrite is applied only under that arm’s guard; conjoining the then-copy with κ and the else-copy with $\neg \kappa$ removes 16, as it should.

When both copies carry the *same* guard ϕ for a word, the disjunction equals $g \wedge \phi$, and `kFOLD` uses that form directly. The difference is decisive for scale. In a Makefile with k independent `ifeq` blocks that each append one object to `obj-y`, every block assigns `obj-y` in one arm; building the disjunction for the untouched words would nest each earlier word’s guard once per later block, so that after 16 blocks the guard of the first object would be a 16-level nested formula rather than

CONFIG_B0=y. We checked with Z3 that the two forms give equivalent formulas for all 14,713 Linux targets of an earlier version of the analysis; they differ syntactically for 2,857 of them. Variables assigned in neither arm keep their entering representation.

3.6 Conditional directory reachability (Algorithm 1, lines 3–16)

The worklist starts from the configured entry directories. For a Makefile reached under ρ , KFOLD restricts its local state by ρ ; directory words in target variables, and the words of `subdir-y` and `subdir-m`, produce child paths with guards (line 14). Paths are resolved to canonical filesystem paths, so a syntactic `./` back edge refers to the same Makefile.

A Boolean visited set would lose conditions at a shared child: the first visit may cover A while a later path adds B . KFOLD instead maintains $R[m]$, the disjunction of guards already propagated to m . For an incoming guard γ it computes $\delta = \gamma \wedge \neg R[m]$ (line 5). If δ is unsatisfiable, the visit is skipped (lines 6–8). Otherwise the cached state of m is restricted by δ (line 12), its targets and children receive that condition, and $R[m]$ becomes $R[m] \vee \gamma$ (line 13). Because the source files and option domains are finite, $R[m]$ can grow only finitely often, so the worklist terminates.

Entry points. Kbuild enters its first directories from top-level Makefiles that KFOLD does not execute, because they consist mostly of rules, recipes, and host-tool invocations. In Linux, the root Kbuild selects `arch/$(SRCARCH)/`, the top-level Makefile adds `lib/` through its `libs-y` list, and `arch/x86/Makefile` adds `arch/x86/pci/` through `drivers-$(CONFIG_PCI)` and builds the boot image with `$(MAKE) $(build)=arch/x86/boot bzImage`. The project settings of KFOLD state these entries: directories, each optionally guarded by an option that must be set, with shell globs for `$(wildcard . . .)` terms; goals such as `arch/x86/boot/bzImage`; fixed definitions such as `SRCARCH=x86` or the guarded `BITS=64?CONFIG_X86_64`; and the objects of the final link. For each subject we copied these entries from the top-level Makefiles; Section 5.1 gives the counts.

3.7 Target families, composites, and aggregation (Algorithm 1, lines 18–23)

Project settings specify target-variable families. Conventional Kbuild uses `obj-` and `lib-`, and Linux also builds the objects of `extra-y` and `always-y`; Barebox adds `pbl-` and `obj-pbl-` for its pre-bootloader, and coreboot uses one family per firmware stage, such as `bootblock-` and `ramstage-`. A target list is a family prefix followed by `y`, `m`, or nothing. Helper variables that merely share the prefix are not targets; Linux’s `obj-pvrusb2-dvb-y`, for example, is a list that the `pvrusb2` Makefile splices into a composite. coreboot’s special families `all-` and `all_x86-` add their objects to several stages; the settings list these aliases, following the handlers in coreboot’s top-level `Makefile.inc`. The extractor records target words ending in `.o` or `.a`, maps source suffixes to object paths for dialects that list sources, and normalizes `..` segments.

Composite resolution follows the members of each selected target t in $t-y$, $t-objs$, and $t-lib$, recursively, and in $t-m$ when t is a module. A member’s condition is the conjunction of its own condition and its container’s condition, as for `accelerator.o` in Table 1. A composite may list an object of its own name, as in `dim-y := dim.o net_dim.o` of `lib/dim`. Members matter at scale: 14,351 of the 29,245 Linux object paths are composite members, such as the fifteen members of `fs/ext2/ext2.o`. The objects of a multi-object host or user program p listed in `hostprogs` or `userprogs` are the words of $p-objs$. When several instances contribute the same path, line 21 disjoins their conditions.

3.8 Kbuild build semantics (Algorithm 1, lines 18–24)

The lists say which objects are *selected*; which object files Kbuild writes depends on further rules of `scripts/Makefile.build`. KFOLD models four of them, each enabled for Linux by a setting.

Built-in composites. When a composite is built into the kernel, Kbuild links its members into the directory’s archive and writes no object for the container. Only a module’s composite, such as `fs/ext2/ext2.o` under `CONFIG_EXT2_FS=m`, has its own object. KFOLD therefore reports a container only through `obj-m`.

Built-in routes. Kbuild builds a directory’s `built-in.a`, and with it the directory’s `obj-y` objects, only if the parent reached the directory through `obj-y` (the `need-builtin` flag). A directory reached only through `obj-m` or `subdir-y` builds its modules, `lib-y`, and `extra-y` objects but not its `obj-y` objects. Under `allmodconfig`, for example, `drivers/video/fbdev/mmp/hw/` is entered through `obj-$(CONFIG_MMP_DISP)` with `CONFIG_MMP_DISP=m`, so its built-in `mmp_ctrl.o` is not built even though `CONFIG_MMP_DISP_CONTROLLER=y`. KFOLD computes, for every Makefile, the guard under which it is reached through `obj-y` entries only, as a fixpoint over the analyzed instances starting from the entry directories, and conjoins it with the conditions of `obj-y` objects.

Rule prerequisites. Some objects are in no list and are built only because a rule needs them. In `arch/x86/entry/vdso/vdso64/`, an included fragment puts `vdso64-image.o` in `obj-y`; Kbuild compiles it from the generated `vdso64-image.c`, a pattern rule `$(obj)/%-image.c: $(obj)/%.so.dbg` generates that file, and an explicit rule links `vdso64.so.dbg` from `$(vobjs)`, which holds `note.o` and the other vDSO objects. KFOLD records each rule’s targets and prerequisites, expanded under their guards, as guarded lists in the state, together with sub-make recipes of the form `$(MAKE) $(build)=dir goal`. The closure at line 24 then starts from every object in Φ and every configured goal and follows explicit rules, pattern rules, the implicit steps from `%.o` to `%.c` and `%.S`, and sub-makes, which it analyzes on demand; every object reached is built under the conjunction of the guards along the way. As in GNU Make, a pattern rule applies only if each prerequisite exists or can be made, so `$(obj)/%-x32.o: $(obj)/%.o` is not applied to a stem that has no object of its own. The boot image, the compressed kernel, the real-mode trampoline, and purgatory are reached this way: the first through the goal `arch/x86/boot/bzImage`, the others through rules and sub-makes in the Makefiles themselves.

Link artifacts. The final link of the kernel writes `vmlinux.o`, `init/version-timestamp.o`, `.vmlinux.export.o` under `CONFIG_MODULES`, and two `.tmp_vmlinux.kallsyms` objects under `CONFIG_KALLSYMS`; the settings list them, following `scripts/link-vmlinux.sh`.

3.9 Meaning of a formula

A formula ranges over the option values of the run’s domain. The Boolean domain contains `y` and `unset`; the tristate domain adds `m` and conjoins `CONFIG_MODULES=y` with every `m` alternative; project settings can add finite domains for other names. Z3 checks guard satisfiability within that domain. Φ_M approximates S_M because the supported language is narrower than GNU Make (Table 2) and because the entry settings stand in for top-level orchestration. Φ_M also omits Kconfig’s `depends on`, `select`, `choice`, and normalization rules. A satisfying assignment of $\Phi_M(o)$ is thus a candidate input for the configurator, not a configuration that Kconfig is known to accept.

4 Implementation

KFOLD is written in Python and uses `pymake3` for parsing and Z3 for guards. Option values are finite Z3 enumeration sorts: a two-value sort `or`, when a project opts in, a three-value sort for `y`, `m`, and `unset`. Guards are built from equalities on these values with conjunction, disjunction, and negation, simplified after each operation; satisfiability queries drive state restriction and worklist propagation. KFOLD serializes each Makefile’s executed state, including its recorded rules, into a per-run cache keyed by canonical path, so a later directory route can restrict the same state under a new guard without parsing the file again (Algorithm 1, line 12). The number of analyzed instances can therefore exceed the number of unique Makefiles. The runner also records the includes spliced

or left unresolved and the files that failed to parse, and for each object path the Makefiles and variables or rules that contribute it.

The project settings file lists the entry directories and goals, target families, family aliases, fixed definitions, link artifacts, and option domain, and switches the built-in semantics of Section 3.8. For Linux it has seven entry directories (the root, `lib/`, and five `arch/x86` directories, three of them guarded), one goal, and the definitions of `SRCARCH` and `BITS`. A sandbox mode, used by the web interface, replaces `$(shell ...)` by the empty string and keeps include splicing inside the analyzed tree.

4.1 Developer commands

A command-line interface answers questions from a cached whole-tree analysis. The cache stores every object's condition, kind, and contributing Makefiles, keyed by a digest of the analyzer and its options, and records the Makefiles and included files the analysis read, so that editing any of them invalidates it.

`why` takes an object and a `.config` and reports whether the object is built, together with its condition chain: the directory route by which Kbuild reaches its Makefile, the target-list line or composite that selects it, and each conjunct's value under the configuration. For an object that is not built, it names the first failing conjunct and the options responsible.

`config-for` takes a patch or a list of files, maps the touched `.c` and `.S` files to objects, and searches for a configuration that builds all of them. It conjoins the objects' conditions Φ_M with a Kconfig model K restricted to the cone of options on which they depend, built with `kconfiglib` and extended for Kconfig syntax newer than that library. K bounds each option from above by its dependencies and from below by its selects, and allows it to be on only through a select, an imply, a visible prompt, or an active default; it over-approximates Kconfig, because it does not decide which default applies. Z3 finds an assignment that changes as few options of a base `.config` as possible, and `config-for` emits it as a fragment. An object whose condition is unsatisfiable together with K , or that conflicts with another touched object, is reported with the reason. With verification, the fragment is applied with `make olddefconfig`, and `config-for` checks that every requested option survives and that `KFOLD` predicts every touched object under the result.

`blindspots` evaluates every condition under a set of configurations and reports the objects that none of them builds, grouped by the `MAINTAINERS` entry that owns the source file and labeled with a likely reason, such as an option of another architecture or an option that none of the configurations enables.

5 Evaluation

We evaluate whether `KFOLD`'s conditions predict the objects that real builds produce, how it scales, whether its conditions support developer tasks, and how it compares with `KMAX` and with itself when design elements are removed. We ask four research questions:

RQ1 (build agreement) When the conditions are evaluated under a configuration, how many predicted objects are compiled, and how many compiled objects are predicted?

RQ2 (scalability) How do `KFOLD`'s time and memory grow with the number of conditional assignments, compared with a forking symbolic executor and with `KMAX`, and what does a whole tree cost?

RQ3 (developer tasks) Do the conditions explain why an object is or is not built, produce configurations that compile the files a real patch touches, and find code that standard configurations never build?

Table 3. Subjects, configurations, and the number of .o files in each fresh build.

Subject	Version	Configuration	Objects built
Linux	7.2.8	tinyconfig (i386)	528
Linux	7.2.8	defconfig (x86-64)	3,133
Linux	7.2.8	Debian (x86-64)	17,191
Linux	7.2.8	allmodconfig (x86-64)	29,625
BusyBox	1.38.0	defconfig	623
Barebox	2026.09.0	sandbox_defconfig	502
Das U-Boot	2026.07	sandbox_defconfig	1,344
coreboot	26.06	qemu-i440fx	626

RQ4 (comparison and ablation) How do KMAX’s conditions compare with KFOLD’s against the same builds, and how much does each design element contribute?

5.1 Subjects, builds, and setup

The subjects are the latest releases of five Kbuild-based systems (Table 3): Linux 7.2.8, BusyBox 1.38.0, Barebox 2026.09.0, Das U-Boot 2026.07, and coreboot 26.06. Linux and BusyBox are the two systems in KMAX’s evaluation [3]; Barebox, U-Boot, and coreboot are bootloader and firmware projects whose Kbuild dialects add families for stages or pre-bootloader code. Each is an unmodified release tarball, checked against the published SHA-256 where the project publishes one. Linux has four configurations: i386 tinyconfig, x86-64 defconfig, the configuration of the Debian kernel package, and x86-64 allmodconfig; the others have one each. Each subject’s settings copy the directory lists and goals of its top-level Makefiles, with each entry annotated with the lines it comes from; for Linux they name seven directories, three of them guarded, one goal, and the definitions of SRCARCH and BITS.

We built every configuration from scratch with `make -k`, from 2.6 minutes for tinyconfig to 100.2 for allmodconfig, and took as the inventory B_c every .o file in the build tree except the *.mod.o files that modpost generates. The builds deviate from stock only where the current toolchain or a missing external input requires it: the Debian configuration has its trusted-key lists emptied, allmodconfig has `-Werror` disabled so that new compiler warnings do not abort files, coreboot builds without a payload, and U-Boot’s final image step fails after all objects have compiled. BusyBox’s `networking/tc.c` no longer compiles against current kernel headers; objects that make attempted and failed to compile are reported separately from false positives.

Metrics. KFOLD emits an extracted object-path set U and a condition for each path. For a configuration c , evaluating the conditions yields a predicted set $P_c \subseteq U$. We define $TP = |P_c \cap B_c|$, $FP = |P_c \setminus B_c|$, $FN_U = |(B_c \cap U) \setminus P_c|$, and $O = |B_c \setminus U|$, and report precision $TP/(TP + FP)$ and overlap $TP/|B_c|$. Overlap charges KFOLD for every compiled object it does not extract, including host tools under `scripts/` and `tools/`, which the analyzed Makefiles do not select; FN_U separately counts extracted objects whose condition is false although the object was compiled. To evaluate a condition, the evaluator substitutes the value of every option in the `.config`, treating an option that is absent or “is not set” as unset.

Setup. All runs used an Intel Core i7-7700 with eight logical CPUs and 31 GiB of RAM under Debian Linux, with Python 3.14.7, Z3 4.13.3, GNU Make 4.4.1, and GCC 16.2.0. The Linux analysis uses the tristate domain. Whole-tree times are the median of three runs on an otherwise idle machine. One script regenerates every result, and every number in this section is generated from its output.

Table 4. Predicted versus compiled Linux 7.2.8 objects. O counts compiled objects outside U , with host tools in parentheses. “Targets” is the overlap when composite members are not resolved.

Configuration	Built	Pred.	TP	FP	FN_U	O (host)	Prec.	Overlap	Targets
tinyconfig (i386)	528	512	512	0	0	16 (16)	100.0%	97.0%	86.6%
defconfig (x86-64)	3,133	3,085	3,085	0	0	48 (48)	100.0%	98.5%	47.2%
Debian (x86-64)	17,191	17,100	17,100	0	0	91 (91)	100.0%	99.5%	35.7%
allmodconfig (x86-64)	29,625	29,556	29,556	0	0	69 (69)	100.0%	99.8%	47.0%

Table 5. Predicted versus compiled objects for the other subjects. “Failed” counts predicted objects that make attempted but could not compile; they are not counted as false positives.

Subject	Built	Pred.	TP	FP	Failed	FN_U	O (host)	Prec.	Overlap
BusyBox	623	590	589	0	1	0	34 (4)	100.0%	94.5%
Barebox	502	462	462	0	0	0	40 (32)	100.0%	92.0%
Das U-Boot	1,344	1,214	1,214	0	0	0	130 (115)	100.0%	90.3%
coreboot	626	477	477	0	0	0	149 (140)	100.0%	76.2%

5.2 RQ1: Build agreement

On all four Linux configurations, every object that `KFOLD` predicts is compiled, and every compiled object that `KFOLD` extracts is predicted (Table 4): there is no false positive and no in-set miss, from 512 predicted objects for `tinyconfig` to 29,556 for `allmodconfig`. The predictions include the boot image, real-mode, `vDSO`, and `EFI-stub` objects, composite members of modules, and the link artifacts. Overlap is 97.0–99.8%. Every compiled object outside U is a host tool under `scripts/` or `tools/`, which the analyzed Makefiles do not select, so without host tools overlap is 100.0% on all four configurations. The predictions include objects that Makefiles create through `$(eval $(call . . . ))`, such as the Intel and Lenovo platform drivers, and `kernel/trace/undefsyms_base.o`, which only a pattern rule for an `always-y` target needs. The last column shows that composite members are essential: counting only the words of target lists, `KFOLD` covers 35.7–86.6% of each inventory.

The other dialects agree as closely (Table 5): no subject has a false positive or an in-set miss. The one `BusyBox` prediction that is not compiled is `networking/tc.o`, whose compilation fails. Without host tools, overlap is 95.2% for `BusyBox`, 98.3% for `Barebox`, 98.8% for `U-Boot`, and 98.1% for `coreboot`. The rest are `BusyBox`’s `built-in.o` archives, which its older `Kbuild` writes as `.o` files; `Barebox`’s link intermediates and generated blobs, such as its default environment; `U-Boot`’s `EFI` example applications, which dedicated rules build; and `coreboot`’s mainboard objects, which its top-level Makefile adds through variables that our entry settings do not copy, together with objects built from the generated devicetree and as relocatable modules.

5.3 RQ2: Scalability

Controlled growth. To isolate the effect of the representation, we generated Makefiles with k independent options in four families: k computed-name appends (`obj-$(CONFIG_Ai) += ai.o`); k `ifeq` blocks that each append to `obj-y`; k `ifeq/else` blocks that set a variable used in an object name (*value*); and k `ifeq` blocks that append to a helper list later appended to `obj-y` (*indirect*). For k from 1 to 128 we ran `KFOLD`, the forking executor that `KFOLD`’s per-word representation replaced (the same code base before that change, which forks at each conditional and merges identical paths), and `KMAX` in both of its output modes, with a limit of 300 s per run.

Table 6. Controlled growth on generated Makefiles with k independent options: states and seconds of the forking executor per family, and the slowest **KFOLD** and **KMAX** (text mode) run over all four families. Times include about 0.5 s of start-up. “t/o” marks the 300 s limit and “–” a run skipped after a smaller k timed out.

k	Forking: states / time (s)			KFOLD	KMAX
	<i>ifeq</i>	<i>value</i>	<i>indirect</i>	time (s)	time (s)
1	2 / 0.51	2 / 0.51	2 / 0.50	0.66	0.76
2	3 / 0.58	8 / 0.51	5 / 0.52	0.62	0.69
4	5 / 0.60	64 / 0.62	33 / 0.52	0.62	0.69
8	9 / 0.69	2,048 / 3.19	1,025 / 0.86	0.64	0.69
12	13 / 0.97	49,152 / 77.75	24,577 / 8.70	0.63	0.71
16	17 / 1.50	t/o	524,289 / 187.09	0.65	0.72
20	21 / 2.54	–	t/o	0.63	0.71
24	25 / 4.31	–	–	0.63	0.72
32	33 / 12.28	–	–	0.62	0.73
48	49 / 68.35	–	–	0.62	0.76
64	65 / 268.37	–	–	0.63	0.79
128	t/o	–	–	0.73	0.92

Table 7. Whole-tree analysis: median time of three runs and peak resident memory. Object paths include composite members, programs, and rule prerequisites.

Subject	Makefile instances	Object paths	Time (s)	Memory (MB)
BusyBox	30	625	0.29	69
Barebox	191	1,770	2.69	70
Das U-Boot	319	3,750	4.85	78
coreboot	513	5,444	22.99	106
Linux	2,215	32,829	143.13	204

Table 6 shows that **KFOLD**’s cost does not grow with k : all 48 runs take 0.61–0.73 s, most of it interpreter start-up, and at most 67 MB, and every run reports exactly the expected objects. The forking executor’s cost depends on how many distinct values the conditionals create. On *append*, each computed-name assignment forks on one option only, and on *ifeq* it merges the arms into $k + 1$ states, yet its time still grows to 268.4 s at $k = 64$ and exceeds the limit at $k = 128$. When each conditional sets a value used in a name or extends a helper list, the states multiply: 49,152 states at $k = 12$ for *value*, and 524,289 states and 187.1 s at $k = 16$ for *indirect*; the next sizes time out. **KMAX** stays below 0.92 s in its text output mode and below 6.78 s in the SMT mode that **kmaxall** uses, so this benchmark separates per-word guards from path forking rather than **KFOLD** from **KMAX**. On *value*, both **KMAX** modes report k objects that no build can produce, such as `c0_$(V0).o`, besides the $2k$ correct ones.

Whole trees. Table 7 gives **KFOLD**’s cost on the five subjects. For Linux, it analyzes 2,215 Makefile instances and extracts 32,829 object paths (16,366 target words and 16,336 composite members, plus programs and rule prerequisites) in 143.1 s with 203.9 MB and 107,208 Z3 checks; the rule closure and built-in routes are included. The other subjects take 0.3–23.0 s. The analysis runs once per tree revision; the developer commands of **RQ3** reuse its cached result. On the same Linux tree, **KMAX** takes 6,275 s and 27.9 GB (Section 5.5).

5.4 RQ3: Developer tasks

We ran the three commands of Section 4.1 on Linux, using the cached whole-tree analysis.

Explaining a build (why). For each of the four built configurations, we drew a seeded random sample of 50 extracted objects that the build produced and 50 that it did not. For all 400 objects, the verdict of why matches the build. For an object that is not built, why names the first part of the condition chain that fails: a directory route in 184 cases, the target-list line in 63, and composite membership in 20. A directory route explains, for instance, the objects under `drivers/net/wireless/` under `tinyconfig`, where the build never enters that directory whatever the objects’ own lines say.

Configurations for real patches (config-for). We used every commit of the Linux 7.2.7-to-7.2.8 stable update that touches a `.c` or `.S` file, 402 commits, and ran `config-for` on each as a patch against `defconfig`, with verification: the fragment is applied with `make olddefconfig` and every requested option is checked in the result. For 384 commits, `config-for` produces a verified configuration under which `KFOLD` predicts every touched object. In 12 commits no touched file is an object of the x86 tree (for example, selftests and other architectures). In the remaining 6 commits, `config-for` reports a touched object as impossible under `Kconfig` and `Kbuild` together; each of these objects needs an option of another architecture, such as `ARCH_K3` on arm64 or SPARC. In every verified configuration, each option that the fragment sets survives `olddefconfig`. The median query takes 6.7 s including verification. From the 384 selectable commits we drew 40 at random, applied each verified configuration to a copy of the `defconfig` build tree, and compiled the touched objects: they compiled for 40 of the 40 commits. The `Kconfig` model used by `config-for` is checked separately on `tinyconfig`, `defconfig`, `Debian`, `allmodconfig`, and `allyesconfig`: the values that `kconfiglib` computes agree with those that `scripts/kconfig` wrote for all 18,749 options except 1 under `allmodconfig`, and each of the five configurations satisfies K , so K does not exclude any of them.

Blind spots. We asked which extracted objects none of x86-64 `allmodconfig`, `allyesconfig`, and `defconfig` builds. `KFOLD` finds 3,199 of 32,829 objects, owned by 509 `MAINTAINERS` entries. Of these, 349 need an option of another architecture, such as `CONFIG_S390`; 2,186 need an option that none of the three configurations enables, and `blindspots` names it; the remaining 664 have no single such option, among them the containers of built-in composites, which `Kbuild` never writes as files. The query shows maintainers which code the usual test configurations never compile.

5.5 RQ4: Comparison with KMAX and ablation

KMAX. We ran `KMAX` v4.10 the way it analyzes a whole tree: `kmaxall` in tristate mode from Linux’s root `Kbuild` with `SRCARCH=x86`, plus `lib/`, under `python3 -O`. As distributed, `kmaxall` aborts the whole run with a Python 3 `TypeError` when any directory fails, so we ran a copy with that one error-reporting call fixed. `KMAX` took 6,275 s and 27.9 GB of memory; `fs/hostfs/` failed with an internal `KeyError`. It reports 31,751 object paths, 28 of them with unexpanded references such as `arch/x86/kernel/apic/probe_$(BITS).o`. Its output gives each object a condition relative to its own `Makefile` and records each subdirectory’s condition separately; we evaluated both the local conditions and the conditions conjoined with every ancestor directory’s.

Table 8 compares the two tools on the same four builds. `KMAX`’s local conditions predict too many objects on small configurations: on `tinyconfig`, its precision is 31.4%, with many false positives in directories that the build never enters, such as `drivers/acpi/`. Conjoining the directory conditions raises precision but lowers overlap. In both variants, `KMAX` misses between 27 and 9,038 compiled objects that it extracts. On the three x86-64 configurations, most of them are composite members in `KFOLD`’s analysis (8,549 of 8,600 under `allmodconfig` with local conditions): `KMAX` gives a composite’s members the condition that the container’s option is `y`, so members of a composite built as a module are never predicted. On `BusyBox`, which has no composites, the tools agree: both predict 589 compiled objects with no in-set miss; `KMAX` takes 73 s and `KFOLD` 0.3 s.

Table 8. KMAX and KFOLD against the same Linux builds: precision / overlap (%) and compiled extracted objects that are not predicted (FN_U). “Local” uses KMAX’s per-Makefile conditions; “with dirs” conjoins the conditions it records for ancestor directories.

Configuration	KFOLD		KMAX local		KMAX with dirs	
	P / O	FN_U	P / O	FN_U	P / O	FN_U
tinyconfig	100.0 / 97.0	0	31.4 / 78.0	27	96.3 / 58.7	129
defconfig	100.0 / 98.5	0	77.2 / 73.0	634	85.5 / 66.6	834
Debian	100.0 / 99.5	0	88.6 / 56.3	6,138	90.1 / 54.3	6,471
allmodconfig	100.0 / 99.8	0	93.7 / 63.0	8,600	94.6 / 61.5	9,038

Table 9. Ablation on Linux: overlap (%), in-set misses, and false positives with one design element removed.

Configuration	Full KFOLD	No includes	No rule closure	No built-in sem.	
	Overlap	Overlap	Overlap FN_U		FP
tinyconfig	97.0	95.8	89.2	0	5
defconfig	98.5	98.0	95.8	5	161
Debian	99.5	90.8	98.8	10	121
allmodconfig	99.8	94.7	99.4	10	144

Ablation. We removed one design element at a time and repeated the comparison with the builds (Table 9); composite members are covered by the last column of Table 4. Without include splicing, KFOLD loses 1,507 Linux objects, mostly in the AMD and Nouveau GPU drivers, whose Makefiles list their objects in included files, and overlap drops to 90.8% on Debian. Without the rule closure, KFOLD loses 112 objects, the boot image, real-mode, and vDSO objects among them, so overlap drops most on tinyconfig, where they are a large share of a small kernel, and objects that are still extracted but reached only through a rule are missed (FN_U). Without the built-in semantics of Section 3.8, overlap is unchanged but built-in composite containers and the built-in objects of directories reached only through `obj-m` are predicted but never written, adding up to 161 false positives. On the other subjects, only the built-in semantics matter: without them, Barebox gains 4 and U-Boot 9 false positives.

5.6 Threats to validity

Construct validity. Precision and overlap compare path names; neither measures whether a source file builds under an arbitrary configuration. Inventories contain objects that are not ordinary compilation units, such as host tools, BusyBox’s `built-in.o` archives, and generated blobs; the tables list them separately rather than removing them.

Internal validity. We developed several of KFOLD’s semantics, including include splicing, the built-in semantics, and the rule closure, while examining discrepancies on Linux builds, so agreement partly reflects fitting to them. Each addition implements a documented behavior of GNU Make or of Kbuild’s Makefile. `build` and has a regression test on a small Makefile. Most of them were developed on Linux v6.6; running on 7.2.8 exposed two defects in the rule closure, and the objects missed in a first run led us to add `$(eval)` and to follow the rules of `always-y` targets; all of these changes were made before the runs reported here. Likewise, an earlier `config-for` run on the same stable patches exposed three options that did not survive `olddefconfig`; we then made *K* account for prompt visibility, defaults, and tristate comparisons, and checked the change against the five real configurations rather than against the patches. The why samples and `config-for` patches are

drawn independently of `KFOLD`. The entry settings copy directory lists and goals from top-level Makefiles by hand, and a copying error would change the traversal.

External validity. Linux has four configurations, all x86, and each other system one. The `config-for` patches come from one stable update, whose fixes may touch fewer optional drivers than new development does. The results describe this implementation on these subjects; they do not show that `KFOLD` handles every Kbuild dialect or configuration.

6 Discussion

6.1 Using a condition

The most direct use of a condition is explanation: a maintainer can see which Makefile expressions, directory routes, and rules cause an object to be built under a proposed assignment. `KFOLD` records, for every object path, the Makefiles and variables or rules that contribute it; for example, it reports that the boot object `compressed/kaslr.o` is a prerequisite of the compressed kernel image and is built under `CONFIG_RANDOMIZE_BASE`. Because the Linux predictions agree with the builds, the conditions can also answer the reverse question without a build: which objects a configuration change adds or removes. The `why` and `blindspots` commands package these two queries.

6.2 Kconfig feasibility

A Kbuild condition and a Kconfig-feasible condition answer different questions. Let $K(c)$ mean that a configuration c satisfies the project's configuration rules. A buildable witness for object o requires at least $K(c) \wedge \Phi_M(o)[c]$ and a successful build. `config-for` conjoins the two and then checks the result with the real configurator and, in our evaluation, the compiler; the Kconfig model agrees with `scripts/kconfig` on every option of five configurations except 1 under `allmodconfig`. That difference is a select from a member of a Boolean choice whose own dependency is `m`, which `kconfiglib` evaluates as `y`. `blindspots`, by contrast, uses only Φ_M and the given configurations, so an object it reports may be one that no configuration can build; the report says which option is missing, not that the code is dead.

6.3 Constructs outside the model

On Linux, `KFOLD` misses no compiled object other than host tools. On the other subjects, the misses are generated blobs, objects built from generated sources or as relocatable modules, and objects that top-level Makefiles add through variables that the entry settings do not copy, such as `coreboot`'s mainboard directory (Table 2).

6.4 Next steps

Three extensions follow from the evaluation. First, `KFOLD` could execute the directory lists and goals of top-level Makefiles instead of taking them from settings; this requires modeling the `include` of architecture Makefiles and the top-level rules that the settings now mirror by hand. Second, the host tools under `scripts/` and `tools/`, which the top-level Makefile builds through its own rules, could be covered by entering those directories as that Makefile does. Third, a conformance suite of small Makefiles, compared with GNU Make over all Boolean and tristate settings of their options, would test each construct in isolation, including `?:=` and `::=`.

7 Related Work

The closest work extracts configuration conditions from Kbuild Makefiles; neighboring work represents variability in source code or symbolic states, and a third line models Kconfig itself.

7.1 Static analysis of Kbuild and build variability

KMAX [3] is the closest prior analysis. It evaluates Kbuild Makefiles with a family-based static analysis whose abstract values are sets of complete string definitions tagged with reaching configurations, represented as BDDs, with the definitions of one variable mutually exclusive; an append combines previous and new definitions by cross-product. It handles computed variable names, includes, and subdirectories, and its evaluation accounts for every C file in the Linux v3.19 and BusyBox 1.25.0 trees. KFOLD shares the idea of carrying configuration conditions on values but guards each word, which keeps k independent appends at k entries and discards order. It also models build semantics beyond the target lists, such as modular composites, built-in routes, and rule prerequisites, and it is validated differently: we evaluate each formula under concrete configurations against fresh builds. On Linux 7.2.8, KMAX’s whole-tree driver takes 6,275 s and 27.9 GB and misses the members of modular composites, while KFOLD takes 143.1 s and agrees with the builds (Section 5.5).

Nguyen and Nguyen [13] symbolically execute Linux Kbuild Makefiles by forking a path state at each conditional. KFOLD replaces this forking with the per-word guards and branch merge of Algorithm 2, and adds the conditional directory worklist, include splicing, composite members, and build semantics; Section 5.3 measures the difference in state growth. Later work uses Kbuild constraints to localize compilation errors and find unmet dependencies [5, 14]; these applications consume build conditions like those KFOLD produces and combine them with Kconfig constraints.

General build-system analyses predate the Kbuild-specific tools. SYMAKE analyzes Makefiles to support transformations, and MAKAO supports build-system analysis and re-engineering [1, 15]. They target build structure and dependencies rather than per-configuration object selection.

7.2 Variability and symbolic-state representations

Variability-aware parsing and analysis keep alternatives across configurations instead of selecting one preprocessor branch at a time. TypeChef, SuperC, and related work show how such representations support reasoning about highly configurable C code [4, 6, 7, 10]. Kbuild selection is complementary: a source-level analysis still needs to know which translation units enter a build. The build language also has computed variable names, deferred expansion, directory traversal, and rules, so its representation needs differ from those of the C preprocessor.

Symbolic execution and state merging provide the broader design context [8, 9]. Merging avoids one environment per branch combination but moves cost into the symbolic expressions and the merge itself; KFOLD’s rule of keeping an unchanged guard unchanged at a merge is what keeps that cost from growing with the number of conditionals.

7.3 Configuration models and validation

Kconfig is a separate variability language. Work on parsing Kconfig and on configuration defects shows why a Makefile-only condition cannot by itself establish feasibility or dead code [2, 12, 16]. Sampling research studies how small sets of configurations can exercise a configurable system [11]. KFOLD’s formulas could serve such workflows as the build-side input, once combined with a Kconfig model.

8 Conclusion

KFOLD extracts a build condition for every object path in a Kbuild tree by attaching guards to individual words, merging conditional arms into one state, splicing included fragments, propagating only uncovered guards through the directory graph, and following Kbuild’s build semantics through composites, built-in routes, and rules. The representation stays linear where a forking executor explodes, and it analyzes the Linux 7.2.8 x86 tree in 143.1 s. Under four Linux configurations, every

predicted object is compiled and every compiled object that `KFOLD` extracts is predicted, and the same holds on four other `Kbuild`-based systems. The conditions explain the build status of every sampled object, and combined with a `Kconfig` model they produce verified configurations for the files that real stable-kernel patches touch. Its main limitation is that top-level orchestration enters through settings copied from the top-level `Makefiles`; executing those `Makefiles` is the next step.

References

- [1] Bram Adams, Herman Tromp, Kris De Schutter, and Wolfgang De Meuter. 2007. MAKAO: An extensible framework for analyzing and re-engineering build systems. In *Proceedings of the 2007 IEEE International Conference on Software Maintenance (ICSM)*. 517–518.
- [2] Christian Dietrich, Reinhard Tartler, Wolfgang Schröder-Preikschat, and Daniel Lohmann. 2012. A Robust Approach for Variability Extraction from the Linux Build System. In *Proceedings of the 16th International Software Product Line Conference (SPLC)*.
- [3] Paul Gazzillo. 2017. Kmax: Finding All Configurations of `Kbuild` `Makefiles` Statically. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. 279–290. doi:10.1145/3106237.3106283
- [4] Paul Gazzillo and Robert Grimm. 2012. SuperC: Parsing all of C by taming the preprocessor. In *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 339–350.
- [5] Paul Gazzillo, Ugur Koc, ThanhVu Nguyen, and Shiyi Wei. 2018. Localizing Configurations in Highly-Configurable Systems. In *Proceedings of the 22nd International Systems and Software Product Line Conference (SPLC), Challenge Track*. doi:10.1145/3233027.3236404
- [6] Christian Kästner, Paolo G. Giarrusso, Tillmann Rendel, Sebastian Erdweg, Klaus Ostermann, and Thorsten Berger. 2011. Variability-Aware Parsing in the Presence of Lexical Macros and Conditional Compilation. In *Proceedings of the ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*. 805–824.
- [7] Andy Kenner, Christian Kästner, Steffen Haase, and Thomas Leich. 2010. TypeChef: Toward Type Checking `#ifdef` Variability in C. In *Proceedings of the 2nd International Workshop on Feature-Oriented Software Development (FOSD)*. 25–32. doi:10.1145/1868688.1868693
- [8] James C King. 1976. Symbolic execution and program testing. *Commun. ACM* 19, 7 (1976), 385–394.
- [9] Volodymyr Kuznetsov, Johannes Kinder, Stefan Bucur, and George Candea. 2012. Efficient state merging in symbolic execution. In *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 193–204.
- [10] Jörg Liebig, Sven Apel, Christian Lengauer, and Christian Kästner. 2011. Variability-aware static analysis at scale. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*. 105–115.
- [11] Flavio Medeiros, Marcio Ribeiro, Rohit Gheyi, Sven Apel, Christian Kästner, Bruno Ferreira, and Karine Santos. 2016. A comparison of 10 sampling algorithms for configurable systems. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)*. 643–654.
- [12] Sarah Nadi, Thorsten Berger, Christian Kästner, and Krzysztof Czarnecki. 2014. Where do configuration defects come from? An empirical study of Linux Kernel configuration bugs. In *Proceedings of the 36th International Conference on Software Engineering (ICSE)*. 677–688.
- [13] ThanhVu Nguyen and KimHao Nguyen. 2020. Using Symbolic Execution to Analyze Linux `KBuild` `Makefiles`. In *Proceedings of the 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 712–716. doi:10.1109/ICSME46990.2020.00078
- [14] Jeho Oh, Necip Fazıl Yildiran, Julian Braha, and Paul Gazzillo. 2021. Finding Broken Linux Configuration Specifications by Statically Analyzing the `Kconfig` Language. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. doi:10.1145/3468264.3468578
- [15] Ahmed Tamrawi, Tung Thanh Nguyen, Jafar M Al-Kofahi, and Tien N Nguyen. 2012. SYMAKE: A tool for analyzing and transforming `Makefiles`. In *Proceedings of the 2012 ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*. 1–4.
- [16] Reinhard Tartler, Daniel Lohmann, Julio Sincero, and Wolfgang Schröder-Preikschat. 2011. Feature Consistency in Compile-Time-Configurable System Software: Facing the Linux 10,000 Feature Problem. In *Proceedings of the Sixth European Conference on Computer Systems (EuroSys)*. 47–60.

A Reproduction Map

`experiments/run_all.sh` regenerates every result from the pinned release tarballs: `fetch.py` downloads and verifies the sources, `build.py` builds each configuration and records its `.config` in `evidence/configs/` and its object inventory in `evidence/inventories/`, `check_kconfig.py` checks the Kconfig model, `evaluate.py` runs `KFOLD` and the ablations against the inventories (`results/agreement/`), `timing.py` times the whole-tree analyses, `kmax.py` runs and scores `KMAX` (`results/kmax/`), `tools/bench_scaling.py` runs the controlled-growth benchmark, which extracts the forking executor from the repository history, and `devtasks.py` runs the developer tasks (`results/devtasks/`). `paper_numbers.py` then writes every number in Section 5 as a $\LaTeX$ macro, together with the table bodies. `results/manifest.json` and `results/builds.json` record source checksums, toolchain versions, and build status; `experiments/README.md` lists the deviations from stock builds.